

Lab 3: Component Modelling & Architectural Pattern Selection

Sameer M Deshpande | PES1UG24CS610

Architecture Selection

We chose **Layered Architecture** for the **Academic Elective Bidding & Allocation System**. The component diagram shows 10 components arranged in a Presentation Layer (Student Portal, Registrar Dashboard), a Business Layer (Bid Submission & Validation, Allocation Solver, Prerequisite Validator, Timetable Conflict Checker, Notification Service) and a Data Layer (Student Database, Elective Catalog, Allocation Results). The external Timetable System is reached only through the Conflict Checker. Components talk through 14 provided/required interfaces (e.g. IBidService, IPrerequisiteCheck, IConflictCheck, IAllocationStore).

Reason 1: The allocation logic is complex and must be isolated

The Allocation Solver has to combine validated bids, prerequisite results, timetable clashes and seat capacity into one fair matching. In a layered design this logic lives only in the Business Layer. It consumes IValidatedBids, IConflictCheck and ICatalogData, and exposes IAllocationAdmin and IAllocationView upward, so the matching algorithm can be tuned or replaced without touching the UI or the database schema. Validation (Bid Submission, Prerequisite Validator) is separate from allocation, so a bid is always checked once, in the same order, whether it comes from a student or is reviewed by a registrar.

Reason 2: Two user roles, one set of shared rules, and a bounded scope

Students (submit bids, view results) and Registrars (review, override) need different screens but the same business rules. Layering gives each role its own Presentation component while both call one Business Layer, so rules are never duplicated. The system serves a single university with a fixed bidding window and a predictable catalog. Microservices would add deployment and network overhead and struggle with the tight Solver ↔ Validator ↔ Student Records coupling. A plain Client-Server design would concentrate all logic in one server and make the Solver a bottleneck when the window closes. The cost of Layered (some call overhead between layers) is acceptable at this scale.

Security Advantage

Layer boundaries give clear control points. Authentication (university SSO) is enforced once, at the two Presentation components, which are the only user-facing parts. Role-based checks and audit logging sit in the Business Layer: students can only submit and view their own bids, and every registrar override is recorded through IAllocationStore. The UI has no dependency on any Data Layer component, so it cannot query the database directly; all SQL is confined to the Data Layer behind fixed interfaces (parameterised queries), which limits injection and tampering risk.

Performance Benefit

The Solver runs as a single business-layer process using in-process calls to the Prerequisite Validator and Conflict Checker, so the batch run (target: 5,000 bids in under 30 s, NFR-001) avoids network hops between validation steps. The Data Layer can cache the Elective Catalog and prerequisite rules behind ICatalogData and IPrereqRules without changing any other component. Result notifications go through an asynchronous queue (INotify), so the Solver never waits on email or portal delivery. The stateless Presentation tier can also be replicated at peak bidding time while a single Solver and database serve all instances.